TECNOSTOCK

Sistema de gestión de inventario, apartados y ventas

PRODUCTO INTEGRADOR DE APRENDIZAJE

Componentes de Lógica de Negocios

Equipo 5 | Caso 05
Versión integrada para entrega: 1.0
Fecha de preparación: 5 de noviembre de 2026
Integrantes: _____
Docente: _____

Documento técnico basado en los artefactos U2 y U3 del Equipo 5.

Control del documento

Campo	Descripción
Proyecto	TecnoStock - solución académica de lógica de negocio
Equipo	Equipo 5, Caso 05
Curso	Componentes de Lógica de Negocios
Versión	1.0 integrada para PIA
Fecha	5 de noviembre de 2026
Base documental	BRD, User Stories, BPMN, glosario, matriz de trazabilidad y reporte de arquitectura U2/U3 entregados por el equipo.
Estado	Prototipo funcional y documentación generada; despliegue remoto y activación real de GitHub Actions quedan sujetos a configuración del equipo.

Contenido del paquete

- Reporte final: este documento, con análisis, arquitectura, implementación, pruebas y conclusiones.
- Documentación técnica editable: `docs/` contiene BRD, 16 historias originales en Gherkin, BPMN, glosario, trazabilidad, ADRs y manual de API.
- Código fuente: `src/` separa dominio, aplicación, infraestructura y API REST.
- Calidad: `tests/` incluye pruebas unitarias y de integración; `docs/evidence/` guarda resultados de ejercicios ilustrativos.
- Operación: Dockerfile, Docker Compose, `.env.example`, workflow de GitHub Actions, OpenAPI JSON y configuración inicial para Render.

Importante: los nombres, precios, clientes, sucursales y existencias incluidos para la demostración son datos ficticios. No representan información comercial real.

1. Resumen ejecutivo

TecnoStock es una cadena de cinco tiendas dedicada a la venta de productos electrónicos. El caso de negocio identifica diferencias entre el inventario registrado y el físico, el riesgo de vender artículos ya apartados, dificultades para rastrear números de serie y criterios inconsistentes al revisar devoluciones. El proyecto integra las actividades de análisis y diseño realizadas en las Unidades 2 y 3 y las convierte en un prototipo verificable.

La solución implementa consultas de inventario por sucursal, apartados con anticipo, ventas directas, tickets QR, registro de entrega y serie, transferencias entre sucursales, monedero por apartados vencidos y registro de devoluciones sujeto a revisión manual del Supervisor. La arquitectura usa Domain Model/DDD, Specifications combinables, máquinas de estado, eventos de dominio, repositorios, Unit of Work, persistencia relacional y API REST documentada con OpenAPI.

La suite ejecutada durante la preparación contiene 23 pruebas aprobadas y alcanzó 92% de cobertura agregada sobre `src/`, frente al mínimo de 80% solicitado en la rúbrica. Los CSV complementarios se evaluaron por separado: 120 escenarios de Specifications y 20 transiciones del flujo de ejemplo. Estos últimos datos se mantienen identificados como material ilustrativo, no como políticas originales de TecnoStock.

2. Análisis del negocio

2.1 Empresa y contexto

TecnoStock representa una cadena de cinco tiendas de retail electrónico. Sus productos pueden venderse directamente o reservarse mediante un apartado. Las sucursales necesitan consultar existencias propias y de otras ubicaciones, mover mercancía con autorización y registrar la unidad física entregada cuando el producto maneja número de serie.

2.2 Problema identificado

- Diferencias entre inventario del sistema y mercancía física.
- Posibilidad de vender una unidad que ya se encuentra reservada para otro cliente.
- Falta de trazabilidad de la unidad exacta mediante número de serie.
- Aplicación no uniforme de los criterios de devolución.

2.3 Objetivo

Centralizar la disponibilidad y el control de operaciones, protegiendo las unidades apartadas o comprometidas, registrando pagos aprobados y manteniendo trazabilidad desde el apartado o venta hasta la entrega física.

2.4 Alcance

Incluido en el prototipo	Fuera de alcance o pendiente
Inventario por sucursal y SKU; consulta global; apartados; ventas directas; transferencias; ticket QR; entrega seriada; vencimientos; monedero; registro/revisión manual de devoluciones.	Cobros bancarios reales; autenticación/SSO; facturación fiscal; canje del monedero; política automatizada de devoluciones; resolución final de discrepancias; notificaciones externas y scheduler permanente.

2.5 Actores

Actor	Responsabilidad principal
Cliente	Solicita compra o apartado; realiza el pago y presenta el ticket para entrega.
Vendedor	Consulta inventario, atiende al cliente y solicita transferencias.
Cajero	Registra ventas y pagos de apartados.
Supervisor	Autoriza o rechaza transferencias y revisa excepciones/devoluciones.
Almacén	Prepara transferencias, recibe mercancía y registra la salida física/serie.
Sistema	Controla disponibilidad, expiraciones, transiciones, eventos y comprobantes.

2.6 Reglas de negocio implementadas

ID	Regla verificada	Base documental
RN-01	El apartado exige un anticipo mínimo del 20% del total.	US-08 / US-10

ID	Regla verificada	Base documental
RN-02	La reserva provisional dura 15 minutos.	US-08 / US-09
RN-03	El apartado confirmado tiene vigencia de 30 días.	US-10 / US-12
RN-04	Máximo cinco artículos activos por cliente, sumando apartados activos.	BPMN y aclaración del reporte U3
RN-05	Los artículos apartados, comprometidos o reservados para transferencia no están disponibles para otra venta.	BRD / inventario U3
RN-06	Pago aprobado permite emitir ticket QR; pago declinado cancela sin comprometer stock.	US-13 / US-14
RN-07	La salida de producto seriado exige una serie única y correspondiente al SKU.	US-15 / US-16
RN-08	Solo Supervisor autoriza/rechaza transferencias.	US-04
RN-09	Una transferencia con discrepancia no ingresa al inventario destino.	US-07
RN-10	El anticipo de un apartado confirmado vencido se abona al monedero una sola vez.	US-12

Regla pendiente: el BPMN y el diagrama de dominio mencionan 15 días para devoluciones, pero el reporte U3 declara que faltan historias y criterios aprobados. Por eso el prototipo no automatiza ese plazo: registra el caso y solicita revisión manual al Supervisor.

3. Historias de usuario y criterios Gherkin

El archivo `docs/user-stories-gherkin.md` conserva las 16 historias entregadas en U2 y transforma sus criterios a formato Given/When/Then para poder relacionarlos con el código y las pruebas. Se añadieron US-17 y US-18 únicamente como propuestas para cerrar el hueco de devoluciones; están marcadas como pendientes de validación y no se consideran políticas aprobadas.

Grupo	Historias U2	Resultado de negocio
Consultas	US-01, US-02	Consultar disponibilidad local y global.
Transferencias	US-03 a US-07	Solicitar, autorizar, enviar, recibir y registrar discrepancias.
Apartados	US-08 a US-12	Crear apartado, anticipo, vencimiento, liquidación y monedero.
Ventas/entregas	US-13 a US-16	Venta directa, pago, ticket QR, consulta y salida física con serie.
Complementarias propuestas	US-17, US-18	Registrar y resolver devolución de forma manual.

La trazabilidad detallada BRD → historia → regla → componente → prueba se encuentra en `docs/traceability.md`. Los escenarios Gherkin completos están en el archivo de historias dentro del repositorio para que puedan editarse y mantenerse junto al código.

4. Modelado de procesos: BPMN

El BPMN U2 describe lanes para Cliente, Vendedor, Cajero, Supervisor y Almacén. El proceso inicia con la solicitud de compra/apartado, consulta disponibilidad, transfiere mercancía cuando no existe stock local, determina la modalidad de compra, ejecuta el pago, emite un ticket QR y culmina con la entrega y registro de serie cuando aplica.

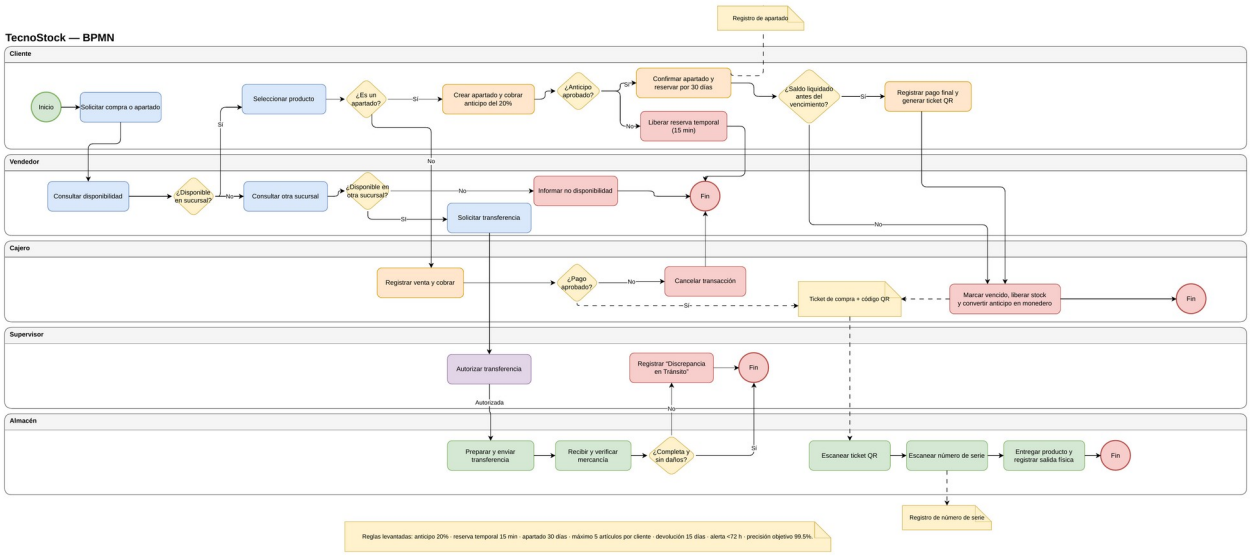


Figura 1. BPMN original de TecnoStock entregado por el Equipo 5 en U2.

Para evitar confusión con políticas no aprobadas, la versión integrada en `docs/BPMN.md` separa el flujo de devoluciones como un subproceso provisional y advierte que su plazo y condiciones requieren validación. El BPMN original se conserva sin alteración en `docs/references/` como evidencia del trabajo previo.

5. Glosario de dominio

El lenguaje ubicuo conecta los nombres utilizados por el negocio con los objetos del código. Los términos principales son:

Término	Significado dentro del sistema
Apartado	Reserva de productos para un cliente con anticipo y vigencia.
Anticipo	Pago parcial mínimo del 20% para confirmar el apartado.
Reserva temporal	Bloqueo provisional de existencias durante 15 minutos.
Stock disponible	Existencia física menos unidades apartadas, comprometidas y reservadas para transferencia.
Ticket QR	Código que permite consultar el detalle de una venta pagada.
Unidad seriada	Pieza física identificada por un número de serie único.
Transferencia	Movimiento de mercancía entre sucursales con autorización y recepción.
Discrepancia en tránsito	Faltante o daño que detiene el ingreso al destino.
Monedero	Saldo a favor generado por un apartado confirmado vencido.
Specification	Objeto que encapsula una condición y permite combinarla con AND, OR y NOT.

Término	Significado dentro del sistema
Aggregate Root	Raíz que protege invariantes de un conjunto del dominio.
Unit of Work	Agrupa escrituras de un caso de uso en commit o rollback.

El glosario ampliado con más de 25 términos y sus relaciones está en `docs/glossary.md`.

6. Arquitectura y diseño

6.1 Enfoque DDD / Domain Model

Se eligió Domain Model como patrón central porque TecnoStock maneja ciclos de vida con reglas temporales, bloqueos de inventario, autorizaciones y trazabilidad física. Transaction Script y Table Module se consideran alternativas útiles para operaciones simples, pero no ofrecen la misma protección de invariantes cuando una operación afecta varios conceptos del dominio.

6.2 Bounded Context y agregados

El contexto principal se denomina Operación de Tiendas TecnoStock. El diseño identifica seis Aggregate Roots: Apartado, InventarioDeSucursal, Venta, Transferencia, MonederoElectronico y Devolucion. Cada uno encapsula operaciones y valida que el estado resultante sea permitido.

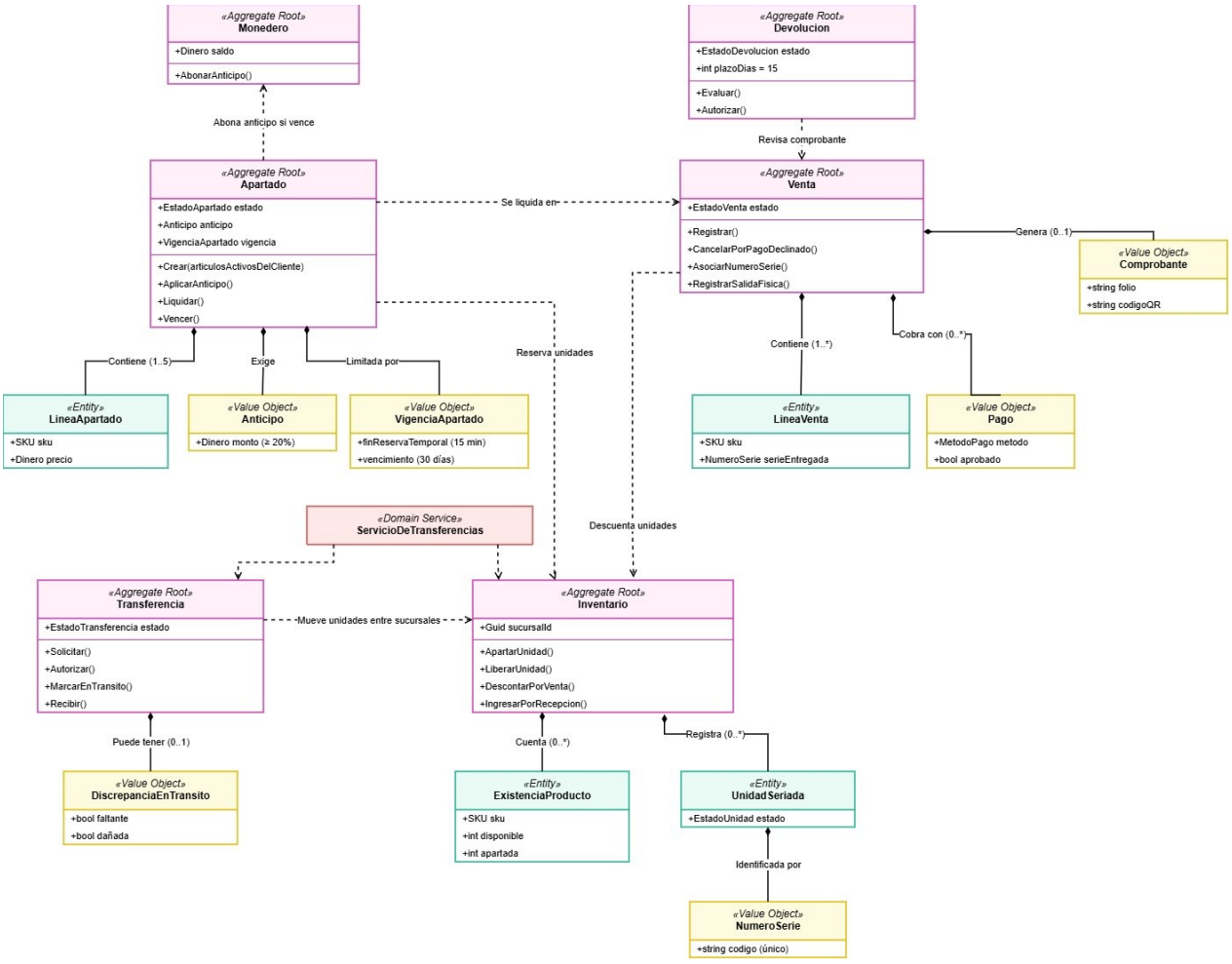


Figura 2. Mapa de dominio U3 del Equipo 5. El atributo de 15 días en Devolución se conserva como referencia histórica, no como política automatizada del prototipo.

6.3 Diagrama de clases

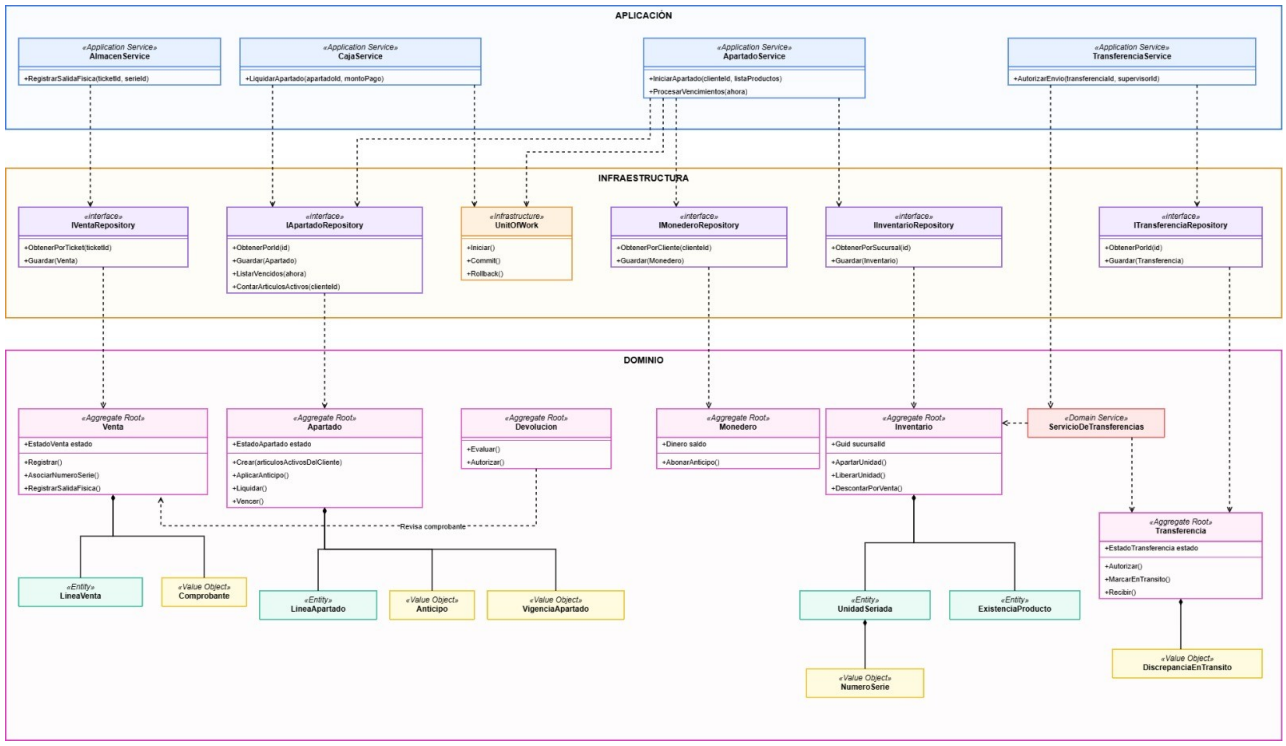


Figura 3. Diagrama de clases U3. El código final separa dominio, aplicación, infraestructura y API.

6.4 Entidades y Value Objects

- Entidades/agregados: Apartado, Transferencia, InventarioDeSucursal, Venta, MonederoElectronico y Devolucion.
- Value Objects y tipos de valor: Dinero, estados enumerados, vigencias, SKU, número de serie, comprobante y QR.
- Entidades de persistencia y catálogos: Cliente, Producto, Sucursal, existencias y unidades seriadas. SQLAlchemy se mantiene en infraestructura y no contamina las clases del dominio.

6.5 Eventos de dominio

Los eventos expresan hechos ya ocurridos y se registran junto con la transacción. Entre ellos se incluyen ApartadoCreado, ApartadoConfirmado, ReservaTemporalExpirada, ApartadoLiquidado, ApartadoVencido, SaldoAFavorGenerado, VentaRegistrada, TransaccionCancelada, SalidaFisicaRegistrada, TransferenciaAutorizada, TransferenciaEnviada, TransferenciaRecibida y DiscrepanciaEnTransitoRegistrada.

6.6 Patrones y decisiones técnicas

Patrón/decisión	Uso en TecnoStock	Justificación
Domain Model / DDD	Reglas de los seis agregados.	Protege invariantes del negocio y estados complejos.
Specification	Validaciones de elegibilidad, stock y ejemplos AND/OR/NOT.	Permite combinar criterios reutilizables.
State Machine	Apartados, transferencias y operaciones de ejemplo.	Evita transiciones inválidas.
Repository	Consultas de catálogo, inventario, ventas y transferencias.	Desacopla el dominio de la persistencia.
Unit of Work	Una sesión transaccional por caso de uso.	Agrupa cambios y rollback ante errores.

Patrón/decisión	Uso en TecnoStock	Justificación
CQRS básico	Separación de Queries y Commands.	Mantiene lectura y modificación claramente diferenciadas.
Domain Events	Registro en tabla de auditoría.	Permite rastrear hechos de negocio y futuras reacciones.

6.7 SOLID

Principio	Aplicación
S - Responsabilidad única	API recibe HTTP; aplicación coordina; dominio valida; infraestructura persiste.
O - Abierto/cerrado	Nuevas Specifications se componen sin modificar las anteriores.
L - Sustitución de Liskov	No se crea una jerarquía artificial de productos; se valida el requisito de serie mediante atributos y reglas.
I - Segregación de interfaces	Puertos/repositorios enfocados en catálogo, inventario, ventas y transferencias.
D - Inversión de dependencias	La capa de aplicación depende de contratos/fábrica de Unit of Work y no de SQLAlchemy directamente.

6.8 ADRs

El repositorio contiene cinco Architecture Decision Records: ADR-001 Domain Model; ADR-002 Unit of Work; ADR-003 CQRS básico; ADR-004 límite de cinco artículos por cliente; ADR-005 revisión manual de devoluciones. Cada archivo documenta contexto, decisión, alternativas y consecuencias. ADR-005 se marca expresamente como provisional.

7. Specification y State Machine

7.1 Specification

La clase Specification permite encapsular una condición de negocio y componerla con AND (&), OR (|) y NOT (~). En el dominio se incluyen reglas de cliente activo, operación con líneas, autorización, preferencia del cliente, importe mínimo, promoción y stock suficiente.

Ejemplos incluidos: `CAN_PROCESS_OPERATION = cliente activo AND detalle no vacío AND operación autorizada`; `REFERENCE_BENEFIT = cliente preferente AND total >= 1500 MXN AND NOT promoción activa`. Estas dos reglas provienen del paquete CSV complementario y se usan solo como ejercicios ilustrativos. No se presentan como reglas comerciales originales de TecnoStock.

7.2 State Machine

Máquina	Secuencia principal	Reglas de protección
Apartado	RESERVA_TEMPORAL → APARTADO_CONFIRMADO → LIQUIDADO o VENCIDO; temporal sin anticipo → CANCELADO.	No se confirma fuera de 15 minutos; no se liquida un apartado vencido.
Transferencia	SOLICITADA → AUTORIZADA → EN_TRANSITO → RECIBIDA o DISCREPANCIA_EN_TRANSITO; SOLICITADA → RECHAZADA.	Solo Supervisor autoriza; solo AUTORIZADA se envía; discrepancia detiene recepción.

Máquina	Secuencia principal	Reglas de protección
Operación ilustrativa	BORRADOR → APROBADA → SURTIDA → ENTREGADA; BORRADOR → CANCELADA.	Máquina separada para ejecutar el ejercicio del CSV; no sustituye el flujo de negocio.

Las transiciones se controlan en `src/domain/state_machine.py`; una transición fuera de la tabla permitida produce InvalidTransition y la API responde con conflicto HTTP. La evidencia completa de transiciones se conserva en `docs/evidence/resultados_transiciones.csv`.

8. Implementación de software

8.1 Tecnologías

Componente	Tecnología
Lenguaje	Python 3.12+
API	FastAPI; documentación automática OpenAPI/Swagger UI
Persistencia	SQLAlchemy 2; SQLite en ejecución local; PostgreSQL en Docker Compose
Validación	Pydantic
Pruebas	Pytest, pytest-cov y HTTPX/TestClient
Contenerización	Docker y Docker Compose
Integración continua	GitHub Actions
Despliegue propuesto	Render mediante `render.yaml`; requiere conectar cuenta/repositorio.

8.2 Capas del proyecto

Carpeta	Responsabilidad	Ejemplos
src/domain	Entidades, reglas, excepciones, Specifications, estados y eventos.	entities.py, specifications.py, state_machine.py
src/application	Casos de uso y puertos.	services.py, ports.py
src/infrastructure	Base de datos, repositorios, seed y Unit of Work.	db.py, repositories.py, unit_of_work.py
src/api	Endpoints, contratos HTTP, QR y creación de aplicación.	routes.py, schemas.py, main.py, qr.py
tests	Pruebas unitarias y de integración.	test_specifications.py, test_state_machine.py, test_*_api.py

8.3 API REST

La API usa el prefijo `/api/v1`. Swagger UI está disponible en `/docs`, Redoc en `/redoc` y OpenAPI en `/openapi.json`. La especificación exportada para el repositorio se encuentra en `docs/api/openapi.json`. Los códigos principales son 200 (consulta/acción), 201 (creación), 404 (recurso inexistente), 409 (conflicto de negocio) y 422 (validación).

Grupo	Endpoints principales
Sistema y catálogos	GET /api/v1/health; GET /sucursales; GET /productos
Inventario	GET /inventario; POST /inventario/ingresos

Grupo	Endpoints principales
Apartados	POST /apartados; GET /apartados/{id}; POST /apartados/{id}/anticipo; POST /apartados/{id}/liquidar; POST /apartados/procesar-vencimientos
Ventas y tickets	POST /ventas; GET /tickets/{qr}; POST /tickets/{qr}/entrega
Transferencias	POST /transferencias; POST /transferencias/{id}/autorizacion; POST /envio; POST /recepcion
Monedero y devoluciones	GET /clientes/{id}/monedero; POST /devoluciones; POST /devoluciones/{id}/resolver
Auditoría	GET /eventos

8.4 Ejecución funcional: recorrido de demostración

1. Consultar `GET /api/v1/inventario?sku=TEC-SSD-001` y mostrar disponibilidad por sucursal.
2. Crear un apartado con `POST /api/v1/apartados` usando `CLI-DEMO-001`, `SUC-01` y una unidad `TEC-SSD-001`.
3. Confirmar el anticipo mínimo del 20% dentro de los 15 minutos con `POST /api/v1/apartados/{id}/anticipo`.
4. Liquidar el saldo antes de los 30 días con `POST /api/v1/apartados/{id}/liquidar`; enseñar el código y PNG QR en base64.
5. Consultar el ticket con `GET /api/v1/tickets/{qr}` y realizar entrega con `POST /api/v1/tickets/{qr}/entrega`. Para una laptop se debe proporcionar la serie exacta; para SSD la estructura de series puede ir vacía porque no requiere serie.
6. Solicitar una transferencia; demostrar que un rol distinto a Supervisor no puede autorizar; autorizar, enviar y recibir desde Almacén.
7. Consultar `/api/v1/eventos` para mostrar el registro de eventos de dominio.

9. Persistencia y configuración

La base relacional contiene catálogos de sucursales, productos y clientes; inventario por sucursal/SKU; apartados y sus líneas; ventas y líneas; unidades seriadas; transferencias; devoluciones y eventos de dominio. El cálculo de disponibilidad descuenta de la existencia física las cantidades apartadas, comprometidas y reservadas para transferencia.

La aplicación lee `DATABASE_URL` como variable de entorno. En ejecución local se propone SQLite (`sqlite:///./tecnostock.db`); Docker Compose conecta a PostgreSQL 16. El archivo `.env.example` muestra las variables y no contiene secretos reales. Las claves predeterminadas de Compose son solo para desarrollo local y deben reemplazarse antes de cualquier exposición pública.

9.1 Docker Compose

El archivo `docker-compose.yml` define tres servicios: `db` (PostgreSQL con healthcheck), `app` (API FastAPI) y `tests` (perfil opcional para ejecutar pruebas y exigir cobertura mínima de 80%). Para iniciar API y base se usa `docker compose up --build -d db app`; para ejecutar pruebas: `docker compose --profile test run --rm tests`. Docker no está instalado en el entorno que generó este reporte, por lo que el archivo está preparado pero el arranque real debe validarse en el equipo.

9.2 CI/CD

`.github/workflows/ci.yml` configura un flujo de integración continua que se ejecuta en push y pull request hacia main/master, instala dependencias y ejecuta pytest con el umbral de cobertura. La ejecución remota

solo se activa después de subir el proyecto al repositorio y realizar push. La configuración está creada; no se afirma que ya exista una ejecución de Actions exitosa en la cuenta del equipo.

9.3 Despliegue

Se incluye `render.yaml` como plantilla inicial para un servicio web Docker y una base PostgreSQL. Un integrante debe conectar el repositorio a Render, revisar el plan/precios vigentes y crear los recursos. Hasta entonces, la URL pública y Swagger en producción permanecen pendientes; la API local sí expone Swagger en `/docs`.

10. Calidad y resultados de pruebas

10.1 Suite automatizada del sistema

Indicador	Resultado medido
Pruebas Python aprobadas	23
Pruebas fallidas	0
Cobertura agregada de `src/`	92%
Umbral solicitado por la rúbrica	80%
Resultado	Supera el umbral en la ejecución local de preparación (5-nov-2026).

La suite cubre Specifications, transiciones de estado, límites de apartados, pagos aprobados/declinados, expiración, monedero, ventas, series, transferencias, discrepancias, devoluciones manuales y endpoints HTTP. Los resultados dependen de la versión del código y se deben repetir al modificar el proyecto.

10.2 Fixtures complementarios del ZIP

Tipo	Evaluados	Resultado
Casos de Specifications	120	57 cumplen y 63 no cumplen las condiciones de referencia.
Transiciones de ejemplo	20	12 aceptadas y 8 rechazadas.

Estos CSV son pruebas ilustrativas descritas como tales en `LEEME.txt`; las reglas de cliente activo/detalle/autorización y cliente preferente/importe/promoción no se agregan al BRD como requisitos originales. Los archivos procesados se guardan en `docs/evidence/`.

10.3 Evidencia

- `coverage.xml` se genera con pytest-cov; no se incluye como garantía permanente del repositorio, debe regenerarse al correr las pruebas.
- `docs/evidence/resultados_casos_prueba.csv` muestra los resultados detallados de los 120 casos ilustrativos.
- `docs/evidence/resultados_transiciones.csv` lista los resultados de las 20 transiciones ilustrativas.
- Los tests usan base SQLite en memoria y reloj controlado para probar vencimientos sin esperar tiempo real.

11. Riesgos, supuestos y trabajo pendiente

Tema	Tratamiento actual	Siguiente validación
Devoluciones	Solicitud con ticket/SKU y resolución manual por Supervisor.	Aprobar plazo, condiciones, accesorios, reembolso y excepciones.
Monedero	Abono único del anticipo cuando vence un apartado confirmado.	Definir canje, vencimiento, combinación y retiro.
Discrepancia en tránsito	Se registra y no ingresa al stock destino.	Definir reclamación, reposición, merma o retorno a origen.
Descuento de inventario	Se compromete disponibilidad al pago/reserva y se reduce existencia física al entregar.	Aprobar el momento contable/logístico definitivo.
Máximo de cinco artículos	Se cuenta la suma de artículos en apartados activos.	Validar concurrencia simultánea con bloqueo/versionado real.
Métodos de pago	El API simula la respuesta mediante `pago_aprobado`.	Definir catálogo y conectar pasarela/terminal real.
Autenticación	El rol Supervisor se recibe en la petición como demo.	Implementar identidad, autorización RBAC y auditoría.
Vencimientos	Endpoint de procesamiento invocado manualmente.	Configurar scheduler, reintentos y monitoreo.

Riesgos no cubiertos para producción: concurrencia multiusuario avanzada, autenticación y permisos reales, pasarela de pagos, facturación, reembolsos, Outbox/broker distribuido, monitoreo, respaldos, rotación de secretos y protección de datos personales.

12. Conclusiones

El PIA integra los artefactos trabajados en las Unidades 2 y 3 en una aplicación con límites de capas y reglas del dominio verificables. Las reglas principales de apartado, disponibilidad, pago, entrega seriada y transferencias se implementan mediante entidades, Specifications, máquinas de estado y operaciones transaccionales. La API REST permite demostrar los flujos con Swagger y la suite automatizada ofrece evidencia medible de calidad.

La principal decisión responsable fue no inventar reglas de devoluciones, uso del monedero ni resolución de discrepancias cuando los documentos previos no las determinan. Estas decisiones quedan registradas para discusión con el stakeholder. El proyecto satisface de forma verificable los elementos que pueden probarse localmente; el equipo todavía debe subir los archivos al repositorio, comprobar el workflow remoto, validar Compose en una máquina con Docker y realizar el despliegue real antes de afirmar que está publicado.

13. Referencias documentales del equipo

- Equipo 5. (2026). BRD_Equipo 5.pdf. Actividad 2, TecnoStock.
- Equipo 5. (2026). User Stories_Equipo 5.pdf. Actividad 2, TecnoStock.
- Equipo 5. (2026). BPMN_Equipo 5.pdf. Actividad 2, TecnoStock.
- Equipo 5. (2026). Glosario_Equipo 5.pdf. Actividad 2, TecnoStock.
- Equipo 5. (2026). Reporte_Actividad 3_Equipo 5_Tecno Stock.pdf. Actividad 3, TecnoStock.
- Equipo 5. (2026). Matriz de Trazabilidad_TecnoStock_Equipo5.pdf. Actividad 3, TecnoStock.
- CLN. (2026). PIA - Proyecto Integrador de Aprendizaje: Componentes de Lógica de Negocios. Instrucciones y rúbrica, 4 páginas.

Anexo A. Archivos clave para la revisión

Archivo	Contenido
README.md	Instalación, ejecución, pruebas, Docker, despliegue y demo.
docs/BRD.md	BRD integrado.
docs/user-stories-gherkin.md	16 historias U2 + 2 propuestas identificadas y escenarios Gherkin.
docs/BPMN.md	Flujo textual, lanes, eventos, gateways y excepciones.
docs/glossary.md	Lenguaje ubicuo del dominio.
docs/traceability.md	Trazabilidad BRD/US/regla/código/prueba.
docs/architecture/	DDD, UML fuente y cinco ADRs.
docs/api/openapi.json	Contrato OpenAPI exportado.
docs/implementation-guide.md	Guía de implementación paso a paso.
tests/	Suite unitaria y de integración.
.github/workflows/ci.yml	Workflow CI.
docker-compose.yml	DB + API + tests.
render.yaml	Configuración inicial de despliegue.
docs/references/	Artefactos originales U2/U3 conservados.